07 · How to Extend OneShot

Where to add things, what each extension must preserve, and a worked example done end to end, with the tests that keep future changes honest.

Extension points at a glance

You want to add…	Touch	Automatically guarded by
a **new failure mode** in the fake provider	`fakeprovider/faults.py` (`FaultMode`), `fakeprovider/server.py` (`_handle_effect_request`)	`test_fault_mode_applies_effect_exactly_as_declared` , the invariant matrix in `tests/test_invariants.py`
a **new action type**	provider endpoint in `fakeprovider/server.py` (`ENDPOINTS`), `oneshot/action_types.py` (`ActionType` , `REGISTRY`), one payload in `tests/test_invariants.py` (`PAYLOADS`)	`test_every_action_type_has_a_demo_payload` (fails until you add the payload), then the full fault-mode matrix
a **new demo scenario**	`demo/scenarios.py` (`@scenario` function using `ctx.expect…`)	`tests/test_demo.py` runs every scenario
a **new outcome rule** (for example honouring `Retry-After`)	`oneshot/provider_client.py` (classification), `oneshot/policy.py` (decision)	`tests/test_provider_client.py` , `tests/test_policy.py` , invariants
a **new storage backend** (for example PostgreSQL)	a class with `ActionStore` 's public methods	`tests/test_store.py` (point the `store` fixture at it), then all integration tests

The rule of thumb: **the provider side declares behaviour, the action type declares capabilities, the policy decides, and the tests iterate over the declarations.** You rarely need to touch the worker, the store or the reconciler.

Worked example: adding `error_after_apply` , end to end

Goal: simulate a provider that commits the charge and *then* hits a bug and returns HTTP 500, a classic real-world failure. This was added in commit `097e29a` exactly as described here.

Step 1: declare the mode and whether it applies the effect

`fakeprovider/faults.py`:

```
     CRASH_AFTER_APPLY = "crash_after_apply"
     """Provider process exits right after committing the effect. Effect APPLIED. Caller sees a
disconnect."""

+    ERROR_AFTER_APPLY = "error_after_apply"
+    """Effect APPLIED, then a bug in the provider returns HTTP 500. Caller sees a server error."""
+
     @property
     def applies_effect(self) -> bool:
-        return self in {FaultMode.SUCCESS, FaultMode.LOST_RESPONSE, FaultMode.SLOW_SUCCESS,
FaultMode.CRASH_AFTER_APPLY}
+        return self in {
+            FaultMode.SUCCESS,
+            FaultMode.LOST_RESPONSE,
+            FaultMode.SLOW_SUCCESS,
+            FaultMode.CRASH_AFTER_APPLY,
+            FaultMode.ERROR_AFTER_APPLY,
+        }
```

Step 2: implement the behaviour

`fakeprovider/server.py`, after the effect has been committed:

```
         if mode is FaultMode.CRASH_AFTER_APPLY:
             srv.log(f"{tag} -> APPLIED {effect.id}, then CRASHING (exit {CRASH_EXIT_CODE})")
             os._exit(CRASH_EXIT_CODE)
+        if mode is FaultMode.ERROR_AFTER_APPLY:
+            # The effect and its idempotency key are already committed, so a
+            # same-key retry will be answered with a replay of the 201.
+            srv.log(f"{tag} -> APPLIED {effect.id}, then answering 500 (bug after commit)")
+            self._send_json(500, {"error": "internal_error"})
+            return
```

Step 3: decide whether OneShot needs to change, and here it does not

Ask: *how does the client classify what this mode produces?* A 500 already maps to `AMBIGUOUS` in `classify_response` (`oneshot/provider_client.py`), and the policy already handles `AMBIGUOUS` per action type: a same-key retry for charges, verification for messages. **No OneShot code changes.** If the new mode produced something not yet classified (for example a new status code), this is where you would add a row to `classify_response` and a case to `tests/test_provider_client.py`. Classify conservatively: when in doubt, use AMBIGUOUS.

Step 4: write the explicit tests

```python
# tests/test_fake_provider.py — the provider contract
def test_error_after_apply_returns_500_but_effect_exists(provider):
    provider.add_faults({"mode": "error_after_apply"})
    assert post(provider, "/v1/charges", CHARGE, key="k1").status_code == 500
    assert len(provider.effects("act_test")) == 1
    retry = post(provider, "/v1/charges", CHARGE, key="k1")
    assert retry.status_code == 201 and retry.headers["Idempotent-Replayed"] == "true"

# tests/test_scenarios.py — OneShot end to end, one test per action type
def test_error_after_apply_charge_is_retried_and_replayed_once(store, client, cfg, provider):
    provider.add_faults({"mode": "error_after_apply"})
    a = submit(store)
    drain(store, client, cfg)
```

```
    assert store.get(a.id).status is S.SUCCEEDED
    assert outcomes(store, a.id) == [Outcome.AMBIGUOUS, Outcome.APPLIED]
    assert len(provider.effects(a.id)) == 1
```

(The message variant asserts `[Outcome.AMBIGUOUS, Outcome.FOUND]`: verified, not resent.)

Step 5: let the invariant suite do the rest

`tests/test_invariants.py` builds its matrix from `list(FaultMode)` and `REGISTRY` at runtime, so three new cases appeared without any further edits:

```
test_fault_mode_applies_effect_exactly_as_declared[error_after_apply]
test_guarantees_hold_for_every_fault_mode_and_action_type[payment.charge-error_after_apply]
test_guarantees_hold_for_every_fault_mode_and_action_type[message.send-error_after_apply]
```

If you had declared `applies_effect` wrongly, or if the mode broke a guarantee for any action type, these would fail. The full, real output of the invariant suite:

```
$ python -m pytest -v tests/test_invariants.py
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

====================================== test session starts ======================================
collecting ... collected 31 items

tests/test_invariants.py::test_every_action_type_has_a_demo_payload PASSED                   [  3%]
tests/test_invariants.py::test_fault_mode_applies_effect_exactly_as_declared[success] PASSED [  6%]
tests/test_invariants.py::test_fault_mode_applies_effect_exactly_as_declared[decline] PASSED [  9%]
tests/test_invariants.py::test_fault_mode_applies_effect_exactly_as_declared[unavailable] PASSED [ 12%]
tests/test_invariants.py::test_fault_mode_applies_effect_exactly_as_declared[timeout] PASSED [ 16%]
tests/test_invariants.py::test_fault_mode_applies_effect_exactly_as_declared[lost_response] PASSED [ 19%]
tests/test_invariants.py::test_fault_mode_applies_effect_exactly_as_declared[slow_success] PASSED [ 22%]
tests/test_invariants.py::test_fault_mode_applies_effect_exactly_as_declared[crash_before_apply] PASSED [
25%]
tests/test_invariants.py::test_fault_mode_applies_effect_exactly_as_declared[crash_after_apply] PASSED [
29%]
tests/test_invariants.py::test_fault_mode_applies_effect_exactly_as_declared[error_after_apply] PASSED [
32%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[success-message.send]
PASSED [ 35%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[success-
payment.charge] PASSED [ 38%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[decline-message.send]
PASSED [ 41%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[decline-
payment.charge] PASSED [ 45%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[unavailable-
message.send] PASSED [ 48%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[unavailable-
payment.charge] PASSED [ 51%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[timeout-message.send]
PASSED [ 54%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[timeout-
payment.charge] PASSED [ 58%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[lost_response-
message.send] PASSED [ 61%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[lost_response-
payment.charge] PASSED [ 64%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[slow_success-
message.send] PASSED [ 67%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[slow_success-
payment.charge] PASSED [ 70%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[crash_before_apply-
message.send] PASSED [ 74%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[crash_before_apply-
payment.charge] PASSED [ 77%]
```

```
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[crash_after_apply-
message.send] PASSED [ 80%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[crash_after_apply-
payment.charge] PASSED [ 83%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[error_after_apply-
message.send] PASSED [ 87%]
tests/test_invariants.py::test_guarantees_hold_for_every_fault_mode_and_action_type[error_after_apply-
payment.charge] PASSED [ 90%]
tests/test_invariants.py::test_guarantees_hold_under_random_fault_sequences[1] PASSED        [ 93%]
tests/test_invariants.py::test_guarantees_hold_under_random_fault_sequences[2] PASSED        [ 96%]
tests/test_invariants.py::test_guarantees_hold_under_random_fault_sequences[3] PASSED        [100%]

==================================== 31 passed in 47.37s =====================================
```

Step 6: add a demo scenario

```python
@scenario(
    "server-error-after-apply",
    "Provider applies the charge, then a bug returns HTTP 500",
    "A 5xx does NOT mean 'nothing happened': ...",
)
def server_error_after_apply(ctx: DemoContext) -> None:
    ctx.arm({"mode": "error_after_apply", "endpoint": "/v1/charges"})
    a = ctx.submit("payment.charge", charge(), "agent-order-1008")
    ctx.run_worker()
    ctx.show(a)
    ctx.expect_status(a, S.SUCCEEDED, 1)
```

Real output of `oneshot demo server-error-after-apply`:

```
$ python -m oneshot demo server-error-after-apply
# captured 2026-11-03 by scripts/capture_outputs.py (exit code 0)

──────────────────────────────────────────── OneShot demo ────────────────────────────────────────────
Each scenario uses a fresh provider process with its own ledger, a fresh OneShot database, and real
worker processes. Faults are injected at the provider (or as failpoints in the worker).
Working directory: $TMPDIR/oneshot-demo-h8bg6688

1/1  server-error-after-apply: Provider applies the charge, then a bug returns HTTP 500 ──────────────
  > fault armed at the provider: error_after_apply on /v1/charges
  > agent POST /actions Idempotency-Key=agent-order-1008 -> HTTP 201 act_6d163d207c11: charge
cus_ada 42.00 USD
  > start worker process
  > worker process finished: every action is SUCCEEDED, FAILED or UNKNOWN (exit 0)
  OneShot's record (event timeline):
  act_6d163d207c11  charge cus_ada 42.00 USD  SUCCEEDED
    +  0.00s (new) -> PENDING  accepted and durably recorded
    +  0.06s PENDING -> IN_FLIGHT  attempt 1: claimed by worker-87204, intent recorded before
sending
    +  0.07s IN_FLIGHT -> PENDING  HTTP 500 (server error; may have applied); outcome unknown ->
retry in 0.30s with the SAME idempotency key (provider deduplicates, so a retry cannot double-apply)
    +  0.42s PENDING -> IN_FLIGHT  attempt 2: claimed by worker-87204, intent recorded before
sending
    +  0.42s IN_FLIGHT -> SUCCEEDED  provider confirmed: replayed original result (provider
deduplicated) as eff_00001
  Provider's side of the story (its log):
    POST /v1/charges ref=act_6d163d207c11 key=act_6d163d207c11 mode=error_after_apply -> APPLIED
eff_00001, then answering 500 (bug after commit)
    POST /v1/charges ref=act_6d163d207c11 key=act_6d163d207c11 -> REPLAY of stored response (no new
effect)
  Provider's independent ledger:
    act_6d163d207c11: 1 effect(s) ['eff_00001']
  [PASS] act_6d163d207c11 is SUCCEEDED and provider ledger has 1 effect(s) (observed: SUCCEEDED, 1)
  Lesson: A 5xx does NOT mean 'nothing happened': the server may have committed before failing.
OneShot classifies it as AMBIGUOUS, and the same-key retry is answered with a replay of the original
success.
```

```
Scorecard (every expectation checked against the
provider's real ledger)
┌──────────────────────────┬────────┬──────┬────────┐
│ scenario                 │ checks │ time │ result │
├──────────────────────────┼────────┼──────┼────────┤
│ server-error-after-apply │    1/1 │ 0.5s │ PASS   │
└──────────────────────────┴────────┴──────┴────────┘
```

Recipe: a new action type

Suppose the agent should also be able to **reserve a table** (`booking.reserve`).

1. **Provider side**: in `fakeprovider/server.py` , write a validator and add an `ENDPOINTS` entry, deciding whether this endpoint honours idempotency keys: `"/v1/bookings": EndpointSpec("booking", honours_idempotency_key=True, validate=_validate_booking)` .
2. **OneShot side**: in `oneshot/action_types.py` , create an `ActionType` with `name` , `provider_path` , the **true** capabilities (`idempotent` , `supports_lookup`), `validate` , `build_request` (it must put the action id in `reference`), and `summary` . Then add it to `REGISTRY` .
3. **Tests**: add a sample payload to `PAYLOADS` in `tests/test_invariants.py` . `test_every_action_type_has_a_demo_payload` fails until you do, and once you have, the new type runs through every fault mode automatically.
4. Optionally add a demo scenario.

> **The one thing you must get right is the capability flags.** They must describe what the provider *really* guarantees. Declaring `idempotent=True` for an endpoint that ignores keys turns every lost response into a duplicate. As a mutation check during development, flipping `message.send` to `idempotent=True` made the invariant suite fail in exactly the `lost_response` , `slow_success` and `crash_after_apply` cases. The recipe above was also followed once, in a throwaway branch, to confirm it works as written (see the README's verification notes).

Recipe: a new storage backend

The worker, API, CLI and reconciler use the store only through these `ActionStore` methods: `submit` , `get` , `list` , `counts` , `claim_next` , `expired_in_flight` , `take_over` , `apply_decision` , `resolve` . A PostgreSQL store would implement the same methods while preserving four properties:

1. **Atomic claim**: select-and-update the next due row in one transaction. PostgreSQL: `SELECT … FOR UPDATE SKIP LOCKED LIMIT 1` , which also lets many workers claim *different* rows concurrently.
2. **Write-ahead attempt**: the attempt row is inserted in the same transaction as the claim.
3. **Lease-checked writes**: `apply_decision` updates only `WHERE lease_owner = :me` , inside the transaction.
4. **Transition whitelist + audit event** in the same transaction as every status change.

Then point the `store` fixture in `tests/conftest.py` at the new class (or parametrize it over both) and run the whole suite. The store tests are the contract.

Recipe: a new demo scenario

Write a function in `demo/scenarios.py`, decorate it with `@scenario(name, title, lesson)`, use `ctx.arm`, `ctx.submit`, `ctx.run_worker(failpoints=…)`, `ctx.show`, `ctx.reconcile`, and state success with `ctx.expect(...)` / `ctx.expect_status(...)`. `tests/test_demo.py` runs every registered scenario, so an expectation that does not hold fails the suite.

What not to do

- Do not add a retry decorator around `ProviderClient.send`; every retry decision belongs in `policy.py`.
- Do not let anything move an action out of `UNKNOWN` except `ActionStore.resolve`.
- Do not make the reconciler rewrite terminal states.
- Do not classify a new error as RETRYABLE unless the provider *guarantees* nothing was applied.

Self-check questions

1. You add a fault mode that returns `202 Accepted` and applies the effect asynchronously 10 s later. Which existing assumption does it violate, and which test would show it?
2. Why did adding `error_after_apply` require no OneShot code change?
3. What breaks if a new action type's `build_request` forgets to include `reference`?
4. Which property must a PostgreSQL `claim_next` keep that `SKIP LOCKED` alone does not give you?

Answers. (1) It breaks the settle-window assumption for non-idempotent types (applied later than `provider_settle_seconds`), and depending on how 202 is classified it may also break G2 (SUCCEEDED before the effect exists). The invariant matrix would catch it: a duplicate or a SUCCEEDED/ledger mismatch. (2) Its observable behaviour, an HTTP 500, was already classified AMBIGUOUS, and the policy handles AMBIGUOUS per capability. (3) Verification and reconciliation look effects up by reference, so they would never find the effect: actions would be resent (duplicates) or flagged as orphans and missing effects. (4) The write-ahead attempt row in the same transaction, plus setting the lease. `SKIP LOCKED` only prevents two workers from picking the same row.